

Blockchain For Developers: Developer Tools

Course Leads:

Sara Magaziotis-Ginori

Olivia Li

Isaac Oh

Aman Shah

Elson Liu

Fujia Wang



BLOCKCHAIN
AT BERKELEY



O Announcements





Homework #1

- Due today, 11:59pm
- Submission link is on the GitHub and Discord

Since it's the first week and students are still enrolling, we'll be lenient on Homework #1's submission!

Try and get it done by next Wednesday, 11:59pm



Homework #2

- Due next Wednesday, 11:59pm
- Already uploaded to GitHub
 - Instructions of deploying an NFT to Ethereum



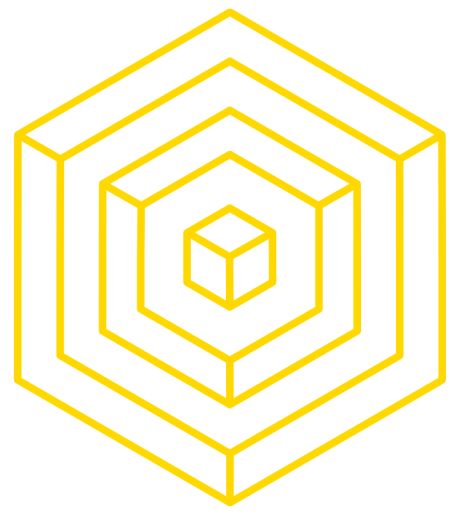
Course GitHub + Discord Channel

<https://github.com/BerkeleyBlockchain/fa25-dev-decal>



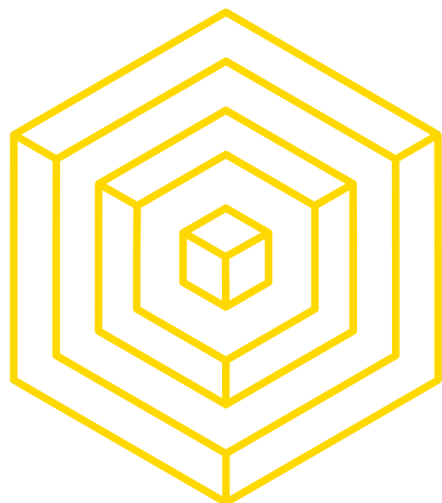
<https://discord.gg/QVbNNts3>



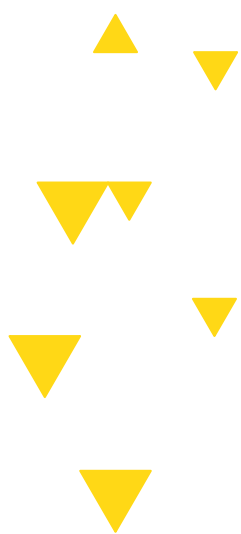


Starting Anonymous Feedback Forms

- We want honest opinions about the content, speed, our teaching, etc etc
- Anonymous feedback form is up on the GitHub



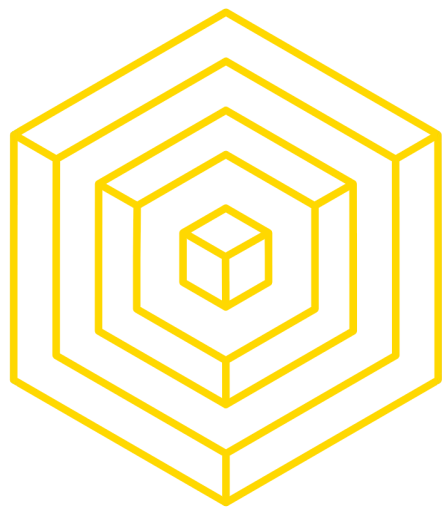
0.5 **Today's Roadmap**



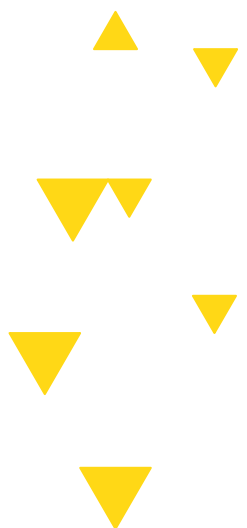


What We're Covering Today + Why

- What: Tools for building on Ethereum
- Why: Ethereum is a **general-purpose blockchain** platform that goes *beyond* simple peer-to-peer payments (like Bitcoin)
 - Ethereum introduced Smart Contracts
 - Smart Contracts - self executing programs that run on the blockchain when certain conditions are met
 - Ethereum Virtual Machine (EVM)
 - Global, decentralized computer that executes code in a secure and deterministic way
 - Ethereum developer ecosystem
 - Largest community of blockchain developers are developing on Ethereum!



Q: What are some problems with deterministic fully public code?





Problems with code on chain

- Front-running and MEV, security risks
 - Attackers can exploit transactions and problems in the code
- Lack of privacy
 - Personal data
- Deterministic code:
 - Everytime you run the code, you need to get the same result, can be hard to bring in off chain data like weather, time, randomness
 - Use oracles, ie Chainlink
- Competitive disadvantage, intellectual properties
- Scalability
 - Deterministic, every node runs code, expensive gas fees. We will talk abt how to optimize
- Updating code and adding features
 - Immutability, cannot patch code. You have to redeploy the contract
 - Old versions remain on chain!!



LECTURE OVERVIEW

- 1 DEVELOPMENT TOOLS OVERVIEW
- 2 METAMASK & ETHERSCAN
- 3 REMIX IDE
- 4 TESTNETS & RPC
- 5 FOUNDRY
- 6 SOLIDITY BASICS



1

DEVELOPMENT TOOLS



DEVELOPMENT TOOLS



METAMASK



REMIX



ETHERSCAN



FOUNDRY

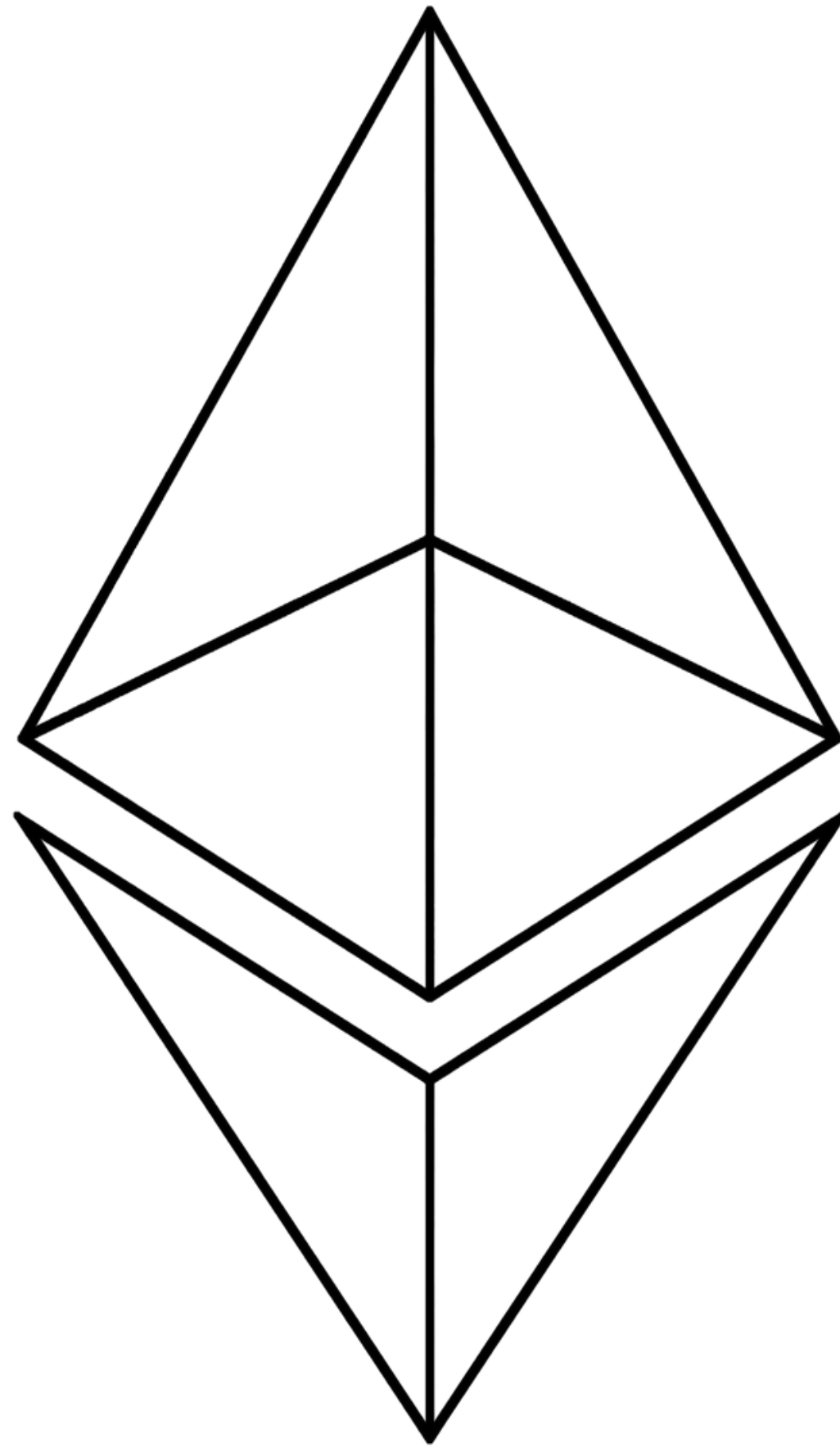


BLOCKCHAIN
AT BERKELEY



DEVELOPMENT TOOLS

...AND GOOD



Mist Blockchain Browser

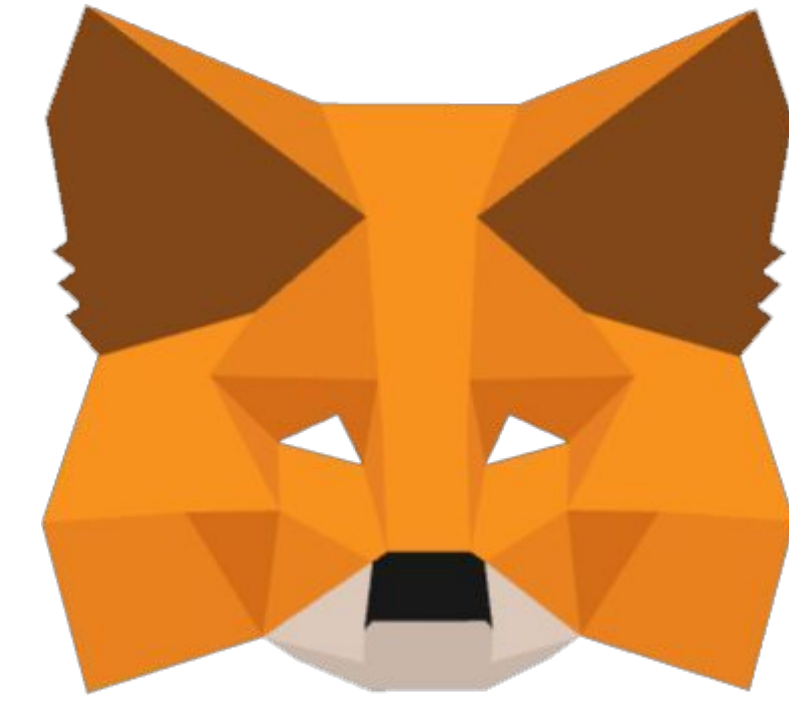
HOMEBREW

Essential macOS package manager

GETH

The Go-Ethereum client

```
brew tap ethereum/ethereum
brew install ethereum
```



METAMASK*

Browser extension; acts as
interface to Dapps

SOLC*

Solidity compiler

```
brew install solidity
```



DEVELOPMENT TOOLS

...AND GOOD



NODE.JS

A useful JavaScript library

NPM

JavaScript package manager

TRUFFLE

Incredibly useful
development environment
and testing framework

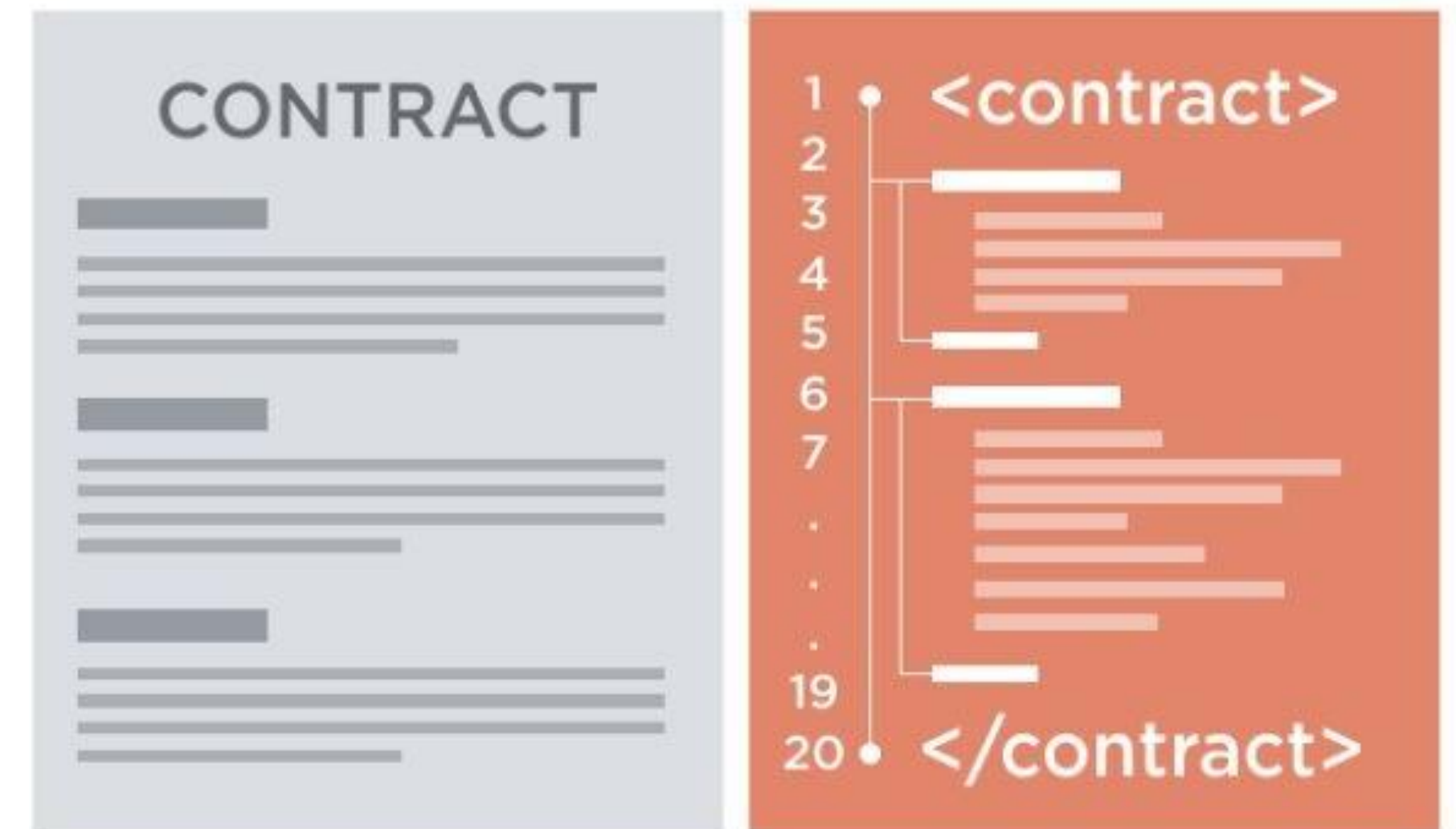
GANACHE/TESTRPC

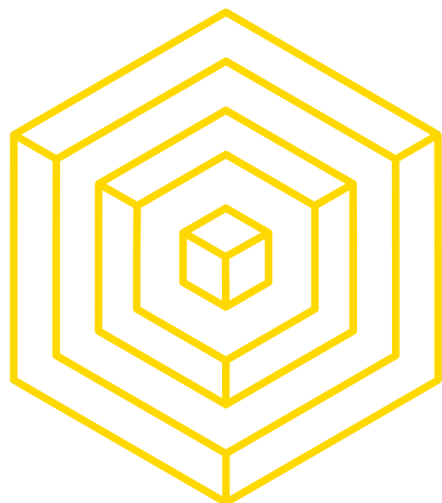
Simulation of full client
behavior



TRUFFLE

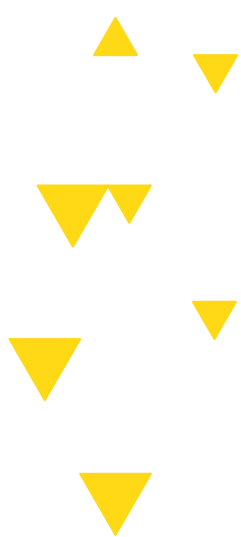
BLOCKCHAIN FOR DEVELOPERS





2

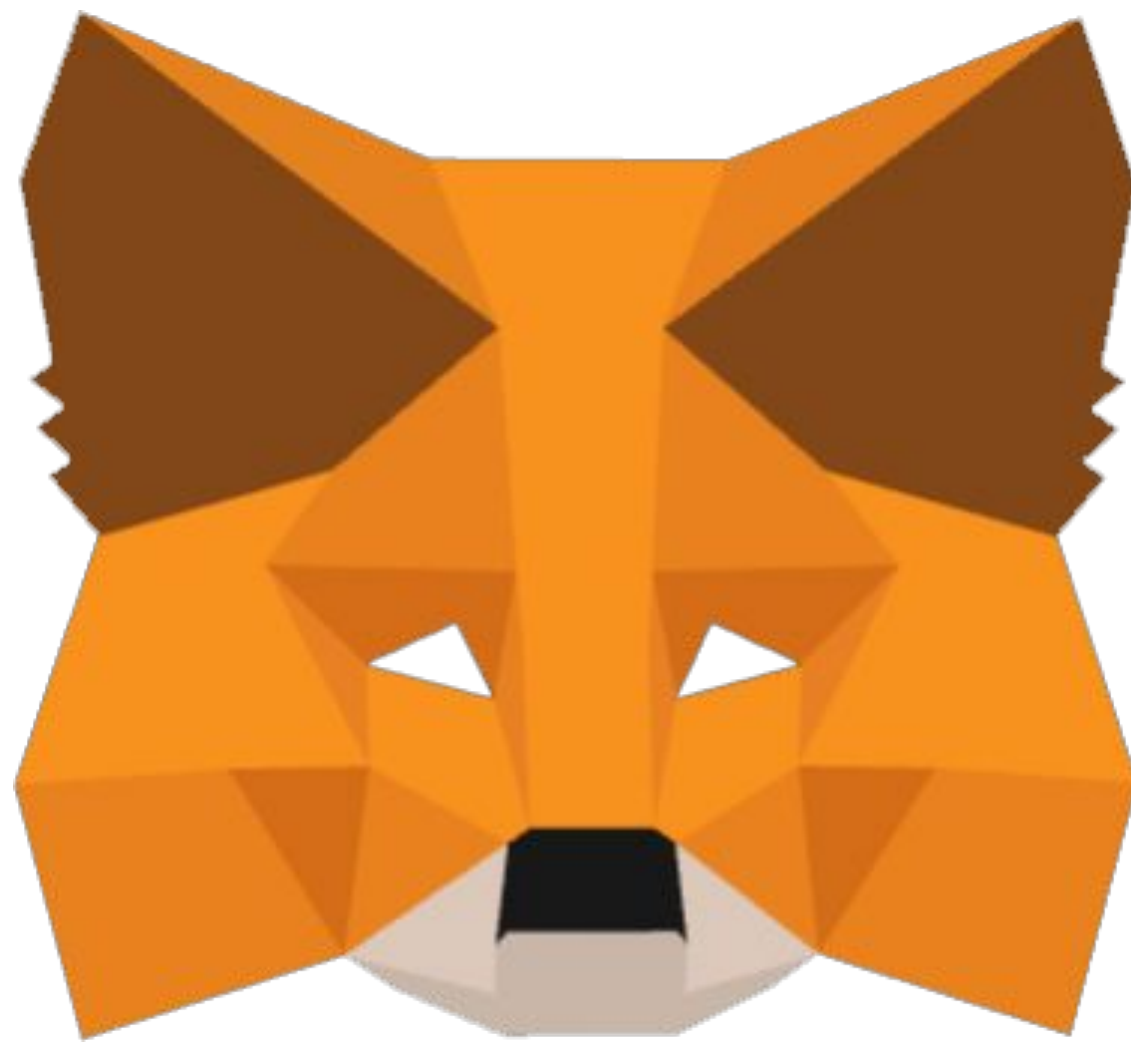
METAMASK & ETHERSCAN





DEVELOPMENT TOOLS

THE MEANS TO ALL EVIL

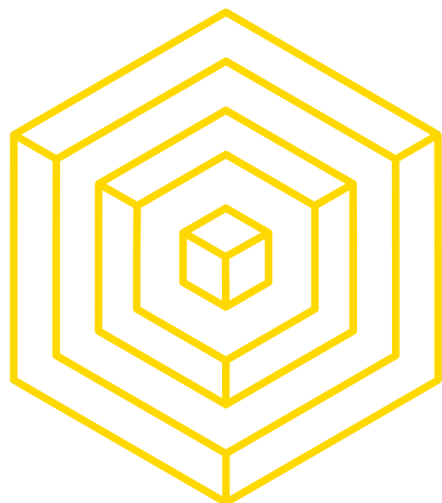


- Chrome extension that allows access to Ethereum-based dApps in a browser
 - Creation and management of identities/accounts
 - Wallet functionality: interface for accounts and transactions
 - there are other wallets like Phantom, Coinbase



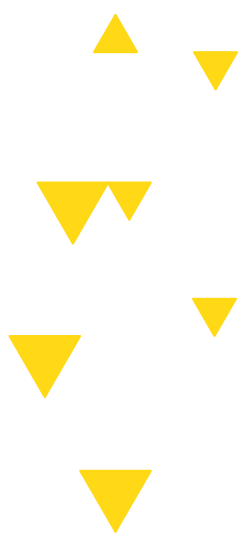
ETHERSCAN Demo

- **sign into my wallet**
- **get public wallet address**
- **past it onto etherscan**
- **everyone can see my past activity and balance**



3

REMIX IDE





```
pragma solidity >=0.7.0 <0.9.0;

import "openzeppelin.sol"; // this import is automatically injected by Remix
import "hardhat/console.sol";
import "../contracts/attendance.sol";

contract AttendanceTest {
    Attendance attendance;

    // Called before each test
    function beforeEach() public {
        attendance = new Attendance();
    }

    // Tests: contract starts with zero student
    function testInitialStudentCountIsZero() public {
        Assert.equal(attendance.getNumberOfStudents(), uint(0), "Initial student count should be zero");
    }

    // Tests: add one student
    function testAddSingleStudent() public {
        attendance.addStudent("Alice");

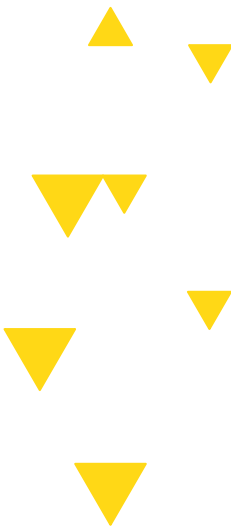
        uint count = attendance.getNumberOfStudents();
        Assert.equal(count, uint(1), "Student count should be 1 after adding Alice");

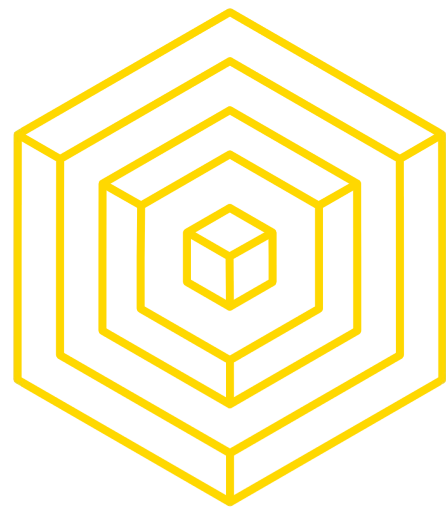
        string[] memory names = attendance.getStudentNames();
        Assert.equal(names.length, uint(1), "Names array length should be 1");
        Assert.equal(names[0], "Alice", "First student should be Alice");
    }

    // Tests: add multiple students
    function testAddMultipleStudents() public {
        attendance.addStudent("Alice");
        attendance.addStudent("Bob");
        attendance.addStudent("Charlie");

        uint count = attendance.getNumberOfStudents();
        Assert.equal(count, uint(3), "Student count should be 3 after adding 3 students");

        string[] memory names = attendance.getStudentNames();
        Assert.equal(names[0], "Alice", "First student should be Alice");
        Assert.equal(names[1], "Bob", "Second student should be Bob");
        Assert.equal(names[2], "Charlie", "Third student should be Charlie");
    }
}
```



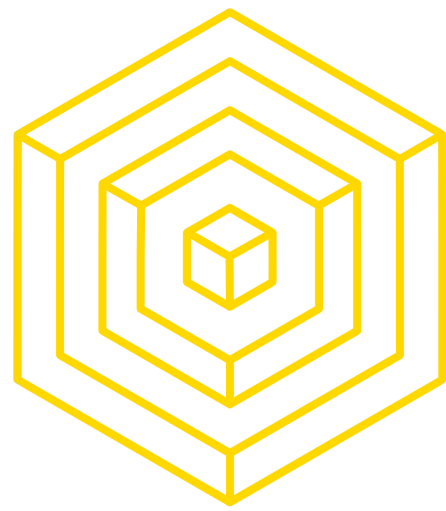


REMIX IDE: Template Project

Create a basic project with necessary file structure

The screenshot shows the REMIX IDE interface with the 'Template Selection' tab active. The 'FILE EXPLORER' on the left shows a project structure with '.deps' and '.prettierrc.json'. The 'Template Selection' panel in the center displays several templates: 'BASIC' (The default project), 'BLANK' (A blank project), 'SIMPLE EIP-7702' (Pectra upgrade allowing externally owned accounts (EOAs) to run contract code.), 'ACCOUNT ABSTRACTION' (A repo about ERC-4337 and EIP-7702), and 'INTRO TO EIP-7702' (A contract for demoing EIP-7702). A yellow arrow points to the 'Create' button for the 'BASIC' template. The 'REMIXAI ASSISTANT' on the right shows a list of topics: 'Fee Handling', 'Withdrawal Pattern', and 'Gas Optimizations'. The bottom status bar includes a 'Scam Alert', 'Initialize as git repo', a 'Did you know?' message, and 'RemixAI Copilot (enabled)'.

AUTHOR: OLIVIA LI



Write your contracts

.sol is a solidity file

FILE EXPLORER

- contracts
- contracts/3_Ballot.sol
- 3_Ballot.sol
- scripts
- deploy_with_ethers.ts
- deploy_with_web3.ts
- ethers-lib.ts
- web3-lib.ts
- tests
- Ballot_test.sol
- storage.test.js
- .prettierrc.json
- README.txt

3_Ballot.sol

```
1 // SPDX-License-Identifier: GPL-3.0
2
3 pragma solidity >=0.7.0 <0.9.0;
4
5 /**
6  * @title Ballot
7  * @dev Implements voting process along with vote delegation
8  */
9 contract Ballot {
10     // This declares a new complex type which will
11     // be used for variables later.
12     // It will represent a single voter.
13     struct Voter {
14         uint weight; // weight is accumulated by delegation
15         bool voted; // if true, that person already voted
16         address delegate; // person delegated to
17         uint vote; // index of the voted proposal
18     }
19
20     // This is a type for a single proposal.
21     struct Proposal {
22         // If you can limit the length to a certain number of bytes,
23         // always use one of bytes1 to bytes32 because they are much cheaper
24         bytes32 name; // short name (up to 32 bytes)
25         uint voteCount; // number of accumulated votes
26     }
27
28     address public chairperson;
29
30     // This declares a state variable that
```

REMIKAI ASSISTANT

- **Fee Handling:** Exact `msg.value` check prevents over/under-payment but could cause UX friction (e.g., if BASE gas fees spike). Consider `>=` with refunds.
- **Withdrawal Pattern:** Current implementation is vulnerable to reentrancy if `to` is a malicious contract. **Fix:**

```
le to) public {
    if (to == address(0)) {
        revert("Invalid address");
    }
    _is().balance;
    amount}(""); // Safer than `trans
    // Add event for transparency
```

4. Gas Optimizations

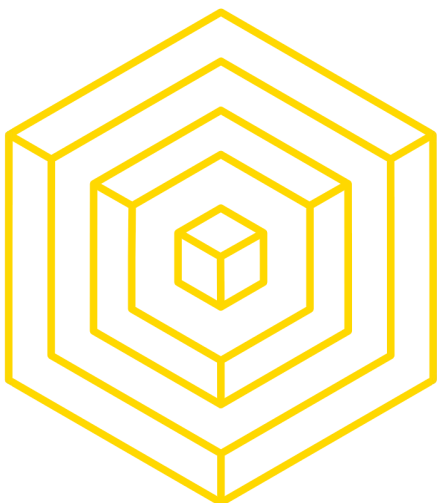
Select Context **Ask** Edit AI Beta

i have written a simple smart contract to

MistralAI Create new workspace with AI

Scam Alert Initialize as git repo Did you know? You can use 'Generate documentation' in the right-click menu to get AI-generated documentation. RemixAI Copilot (enabled)

AUTHOR: OLIVIA LI



REMIXv0.71.0

attendance

olivli14Theme

REMIXAI ASSISTANT

Issue	Solution
"Injected Provider" missing	Ensure MetaMask is installed and unlocked. Refresh Remix.
Deployment fails	Check gas limits in MetaMask (increase by 10–20%).
Transaction pending	Wait or replace with a higher gas fee in MetaMask.
"Contract not found"	Verify the address is correct and the network matches (e.g., Sepolia).

Select Context

AskEditAI Beta

Select context and ask me anything!

MistralAICreate new workspace with AI

Compile

deploy_with_ethers.ts.prettierrc.jsoncontracts/attendance.solattendance_test.sol

```
12 contract Attendance {
13   // Dynamic array of student names
14   string[] private studentNames;
15
16   // Return number of students currently stored
17   function getNumberOfStudents() public view returns (uint) {
18     return studentNames.length;
19   }
20
21   // Return the full list of student names
22   function getStudentNames() public view returns (string[] memory) {
23     return studentNames;
24   }
25
26   // Append a new student name and update the count automatically
27   function addStudent(string memory name) public {
28     studentNames.push(name);
29   }
30 }
```

Explain contract

AI copilot

0Listen on all transactionsFilter with transaction hash or ad...

transact to Attendance.addStudent errored: Error occurred: [object Object].

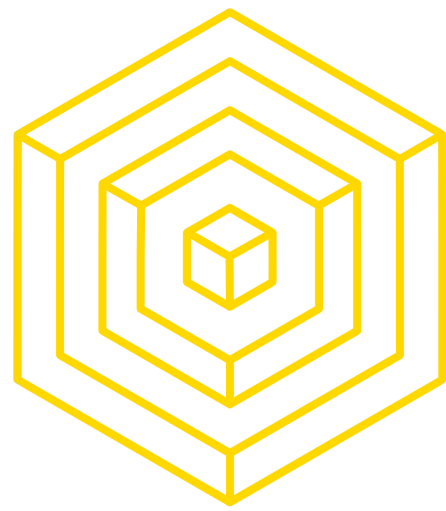
[object Object]

If the transaction failed for not having enough gas, try increasing the gas limit gently.

Scam AlertInitialize as git repo

Did you know? You can use 'Generate documentation' in the right-click menu to get AI-generated documentation.

RemixAI Copilot (enabled)



Test your contracts

write tests in test folder, test before deploying

The screenshot displays the Remix IDE interface. On the left, the 'FILE EXPLORER' panel shows a project structure with folders like '.deps', 'artifacts', 'build-info', 'contracts', and 'scripts'. A yellow box highlights the 'tests' folder, which contains the file 'attendance_test.sol'. A yellow arrow points to this file. The main editor area shows the content of 'attendance_test.sol', which includes a Solidity contract definition and test functions. The code is as follows:

```
1 // SPDX-License-Identifier: GPL-3.0
2
3 pragma solidity >=0.7.0 <0.9.0;
4 import "remix_tests.sol"; // this import is automatically injected by Remix.
5 import "hardhat/console.sol";
6 import "../contracts/attendance.sol";
7
8 contract AttendanceTest {
9     Attendance attendance;
10
11     /// Called before each test
12     function beforeEach() public {
13         // infinite gas
14         attendance = new Attendance();
15     }
16
17     /// Test: contract starts with zero students
18     function testInitialStudentCountIsZero() public {
19         // infinite gas
20         Assert.equal(attendance.getNumberOfStudents(), uint(0), "Initial student count should be 0");
21     }
22
23     /// Test: add one student
24     function testAddSingleStudent() public {
25         // infinite gas
26         attendance.addStudent("Alice");
27     }
28 }
```

Below the code editor, the 'EXPLAIN CONTRACT' panel is visible, showing a welcome message and instructions on how to use the terminal. The terminal output includes:

```
Welcome to Remix 0.71.0

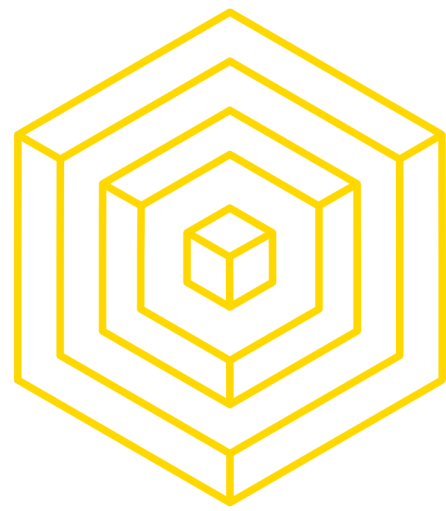
Your files are stored in indexedDB, 407.42 KB / 136.96 GB used

You can use this terminal to:
• Check transactions details and start debugging.
• Execute JavaScript scripts:
  - Input a script directly in the command line interface
  - Select a Javascript file in the file explorer and then run `remix.execute()` or `remix.executeCurrent()` in the command line interface

>
```

The bottom status bar shows a 'Scam Alert' and a 'Did you know?' message: 'You can use 'Generate documentation' in the right-click menu to get AI-generated documentation.'

AUTHOR: OLIVIA LI



Testing your contracts

Press the compile button. Go to Solidity Unit Testing. Run

The screenshot shows the Remix IDE interface. On the left, the 'SOLIDITY UNIT TESTING' panel is active. It contains a 'Test directory' field with 'tests' entered and a 'Create' button. Below this are 'Generate' and 'How to use...' buttons. A yellow arrow points to the 'Run' button (a blue button with a play icon). Below the 'Run' button is a 'Stop' button. Further down, there are checkboxes for 'Select all' and 'tests/attendance_test.sol'. The 'Progress' section shows '1 finished (of 1)' with a 'PASS' status for 'AttendanceTest'. Below this, three test results are listed: 'Test initial student count is zero', 'Test add single student', and 'Test add multiple students', all with green checkmarks. At the bottom of the panel, it says 'Result for tests/attendance_test.sol'. On the right, the code editor shows a Solidity file named 'b3.ts'. The code includes a license header, pragma statement, imports for 'remix_tests.sol', 'hardhat/console.sol', and 'contracts/attendance.sol'. It defines a 'contract AttendanceTest' with a 'beforeEach' function and two test functions: 'testInitialStudentCountIsZero' and 'testAddSingleStudent'. The bottom of the IDE shows a terminal with the 'Welcome to Remix 0.71.0' message and instructions on how to use the terminal.

AUTHOR: OLIVIA LI



Compile Your Contract

press compile

The screenshot displays the Remix IDE interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' sidebar is visible, showing the environment set to 'Injected Provider - MetaMask' on the 'Sepolia (11155111) network'. The account '0xDa8...b9d70' is selected. The gas limit is set to 'Estimated Gas' with a custom value of '3000000'. The value is '0' Wei. The contract 'Attendance - contracts/attendance' is selected, and the EVM version is 'prague'. The 'Deploy' button is highlighted. Below the sidebar, it shows 'Transactions recorded 6' and 'Deployed Contracts 2'. The main editor area shows the Solidity code for the 'Attendance' contract. A yellow box highlights the 'Compile' button in the top toolbar. The code includes a dynamic array of student names, a function to get the number of students, and a function to get student names. A tooltip shows the current line of code: 'contracts/attendance.sol 26:51 memory name) public { infinite gas'. The bottom panel shows the 'Explain contract' section with a search bar and a list of transactions. A transaction is shown with a green checkmark, indicating it was successful. The transaction details include the block number (9225020), transaction index (11), from address (0xda8...b9d70), to address (Attendance.addStudent(string)), value (0 wei), data (0x453...00000), logs (0), and hash (0x342...11644). A 'Debug' button is visible next to the transaction details. The bottom status bar includes a 'Scam Alert' and a 'Did you know?' message about generating documentation.

```
10
11
12 contract Attendance {
13     // Dynamic array of student names
14     string[] private studentNames;
15
16     // Return number of students currently stored
17     function getNumberOfStudents() public view returns (uint) { 2439 gas
18         return studentNames.length;
19     }
20
21     // Return the full list of student names
22     function getStudentNames() public view returns (string[] memory) { infinite gas
23         return studentNames;
24     }
25
26     // Append a new student name and update the count automatically
27     function addStudent(string memory name) public { infinite gas
28
29     }
30
31 }
```

contracts/attendance.sol 26:51 memory name) public { infinite gas

Explain contract

0 Listen on all transactions Filter with transaction hash or ad...

if the transaction failed for not having enough gas, try increasing the gas limit gently.

transact to Attendance.addStudent pending ...

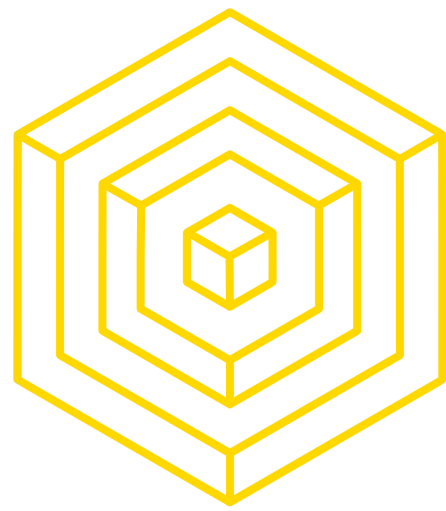
[view on Etherscan](#) [view on Blockscout](#)

[block:9225020 txIndex:11] from: 0xda8...b9d70
to: Attendance.addStudent(string) 0x376...c2f0e value: 0 wei
data: 0x453...00000 logs: 0 hash: 0x342...11644

Debug

Scam Alert Initialize as git repo Did you know? You can use 'Generate documentation' in the right-click menu to get AI-generated documentation.

AUTHOR: OLIVIA LI



Select the Right Contract

The screenshot displays the Remix IDE interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' sidebar is visible. The 'ENVIRONMENT' is set to 'Injected Provider - MetaMask' on the 'Sepolia (11155111) network'. The 'ACCOUNT' is '0xDA8...b9d70 (0.093422...)'. The 'GAS LIMIT' is set to 'Estimated Gas' with a custom value of '3000000'. The 'VALUE' is '0 Wei'. The 'CONTRACT' dropdown is highlighted with a yellow box, showing 'Attendance - contracts/attendance'. Below this, there are buttons for 'Deploy', 'Publish to IPFS', and 'At Address'. The 'Transactions recorded' section shows 6 transactions. The 'Deployed Contracts' section shows 2 contracts.

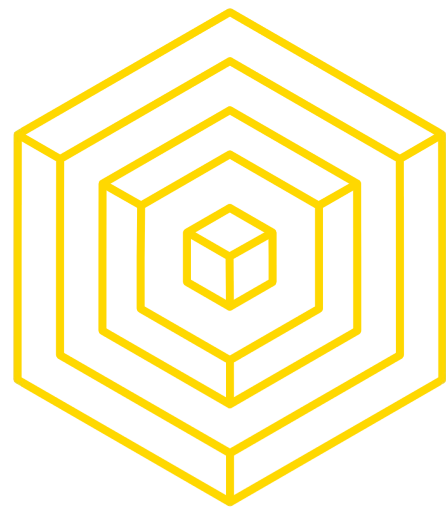
The main editor displays the Solidity code for the 'Attendance' contract. The code is as follows:

```
9  /*
10
11
12  contract Attendance {
13      // Dynamic array of student names
14      string[] private studentNames;
15
16      // Return number of students currently stored
17      function getNumberOfStudents() public view returns (uint) { 2439 gas
18          return studentNames.length;
19      }
20
21      // Return the full list of student names
22      function getStudentNames() public view returns (string[] memory) { infinite gas
23          return studentNames;
24      }
25
26      // Append a new student name and update the count automatically
27      function addStudent(string memory name) public { infinite gas
28          ;
29      }
30
31  }
```

The bottom panel shows the 'Explain contract' section with a search bar and a list of transactions. The first transaction is highlighted:

```
[block:9225020 txIndex:11] from: 0xda8...b9d70
to: Attendance.addStudent(string) 0x376...c2f0e value: 0 wei
data: 0x453...00000 logs: 0 hash: 0x342...11644
```

AUTHOR: OLIVIA LI



Choose Metamask Provider

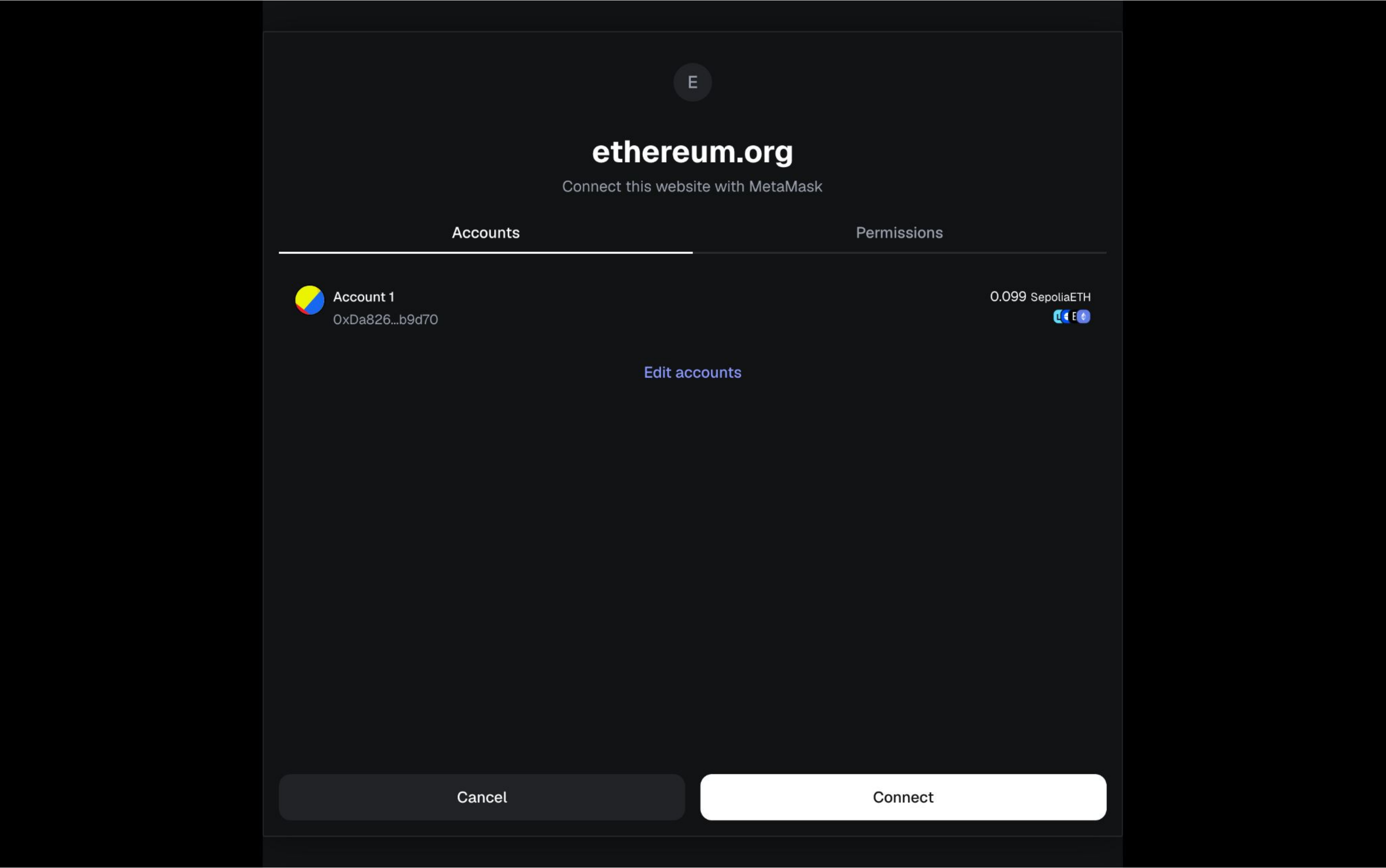
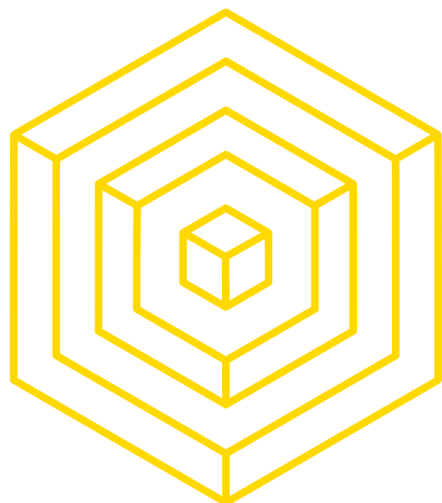
The screenshot shows the REMIX IDE v0.71.0 interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' sidebar is visible. A yellow box highlights the 'Injected Provider - MetaMask' dropdown menu, which is set to 'Sepolia (11155111) network'. Below this, the account '0xDA8...b9d70 (0.093422...)' is selected. The 'GAS LIMIT' is set to 'Estimated Gas' with a custom value of '3000000'. The 'VALUE' is '0 Wei'. The contract 'Attendance - contracts/attendance' is selected, and the 'evm version' is 'prague'. The 'Deploy' button is highlighted. The main editor shows the Solidity code for the 'Attendance' contract. The bottom panel shows the transaction details for 'Attendance.addStudent', indicating a successful deployment on Sepolia.

```
9  /*
10
11
12  contract Attendance {
13      // Dynamic array of student names
14      string[] private studentNames;
15
16      // Return number of students currently stored
17      function getNumberOfStudents() public view returns (uint) {
18          return studentNames.length;
19      }
20
21      // Return the full list of student names
22      function getStudentNames() public view returns (string[] memory) {
23          return studentNames;
24      }
25
26      // Append a new student name and update the count automatically
27      function addStudent(string memory name) public {
28          studentNames.push(name);
29      }
30  }
31
```

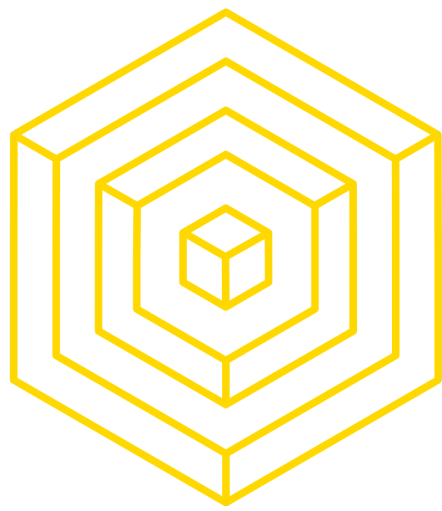
Deploy on Sepolia!!!!

Sepolia is a testnet
aka not real money.
We don't want to
spend real money
testing!!

AUTHOR: OLIVIA LI



AUTHOR:OLIVIA LI



Deploy

put code on blockchain

30

The screenshot displays the REMIX IDE v0.71.0 interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' sidebar is visible, showing the environment set to 'Injected Provider - MetaMask' on the 'Sepolia (11155111) network'. The account '0xDA8...b9d70' is selected. The 'GAS LIMIT' is set to 'Estimated Gas' with a custom value of '3000000'. The 'VALUE' is '0 Wei'. The contract 'Attendance - contracts/attendance' is selected. A yellow box highlights the 'Deploy' button and the 'Publish to IPFS' checkbox. Below this, there are buttons for 'At Address' and 'Load contract from Address'. The bottom of the sidebar shows 'Transactions recorded 6' and 'Deployed Contracts 2'.

The main editor shows the Solidity code for the 'Attendance' contract:

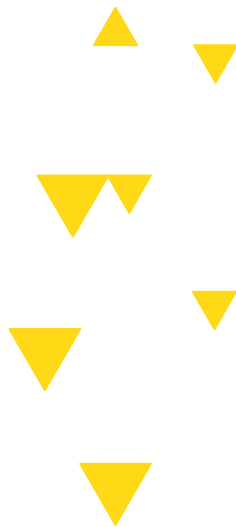
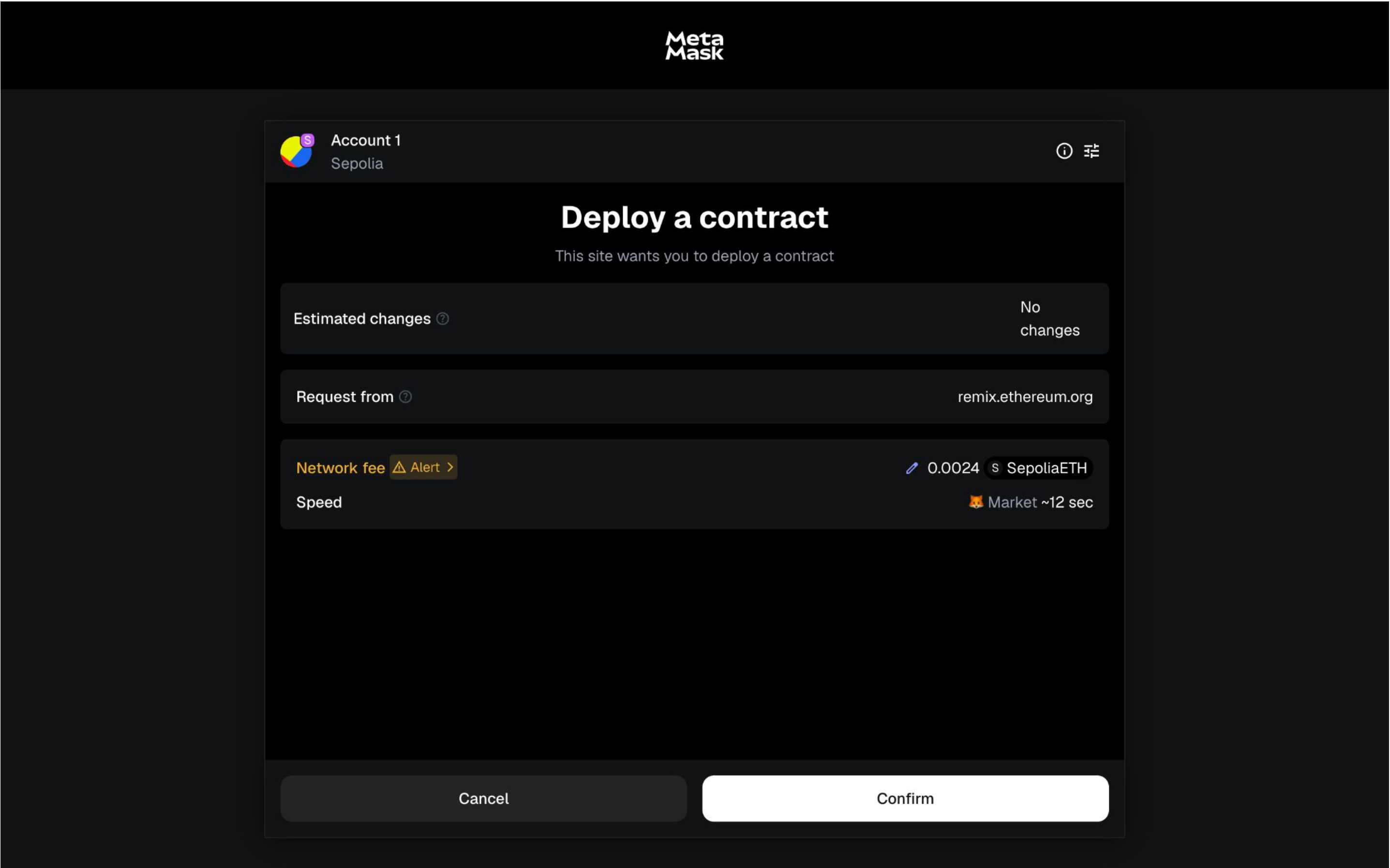
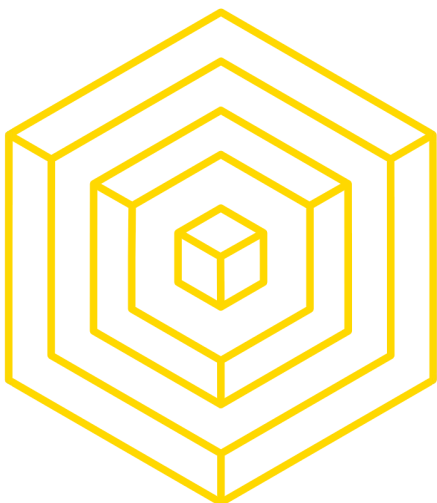
```
9  /*
10
11
12  contract Attendance {
13      // Dynamic array of student names
14      string[] private studentNames;
15
16      // Return number of students currently stored
17      function getNumberOfStudents() public view returns (uint) { 2439 gas
18          return studentNames.length;
19      }
20
21      // Return the full list of student names
22      function getStudentNames() public view returns (string[] memory) { infinite gas
23          return studentNames;
24      }
25
26      // Append a new student name and update the count automatically
27      function addStudent(string memory name) public { infinite gas
28          ;
29      }
30  }
31
```

The bottom panel shows the transaction details for the deployment:

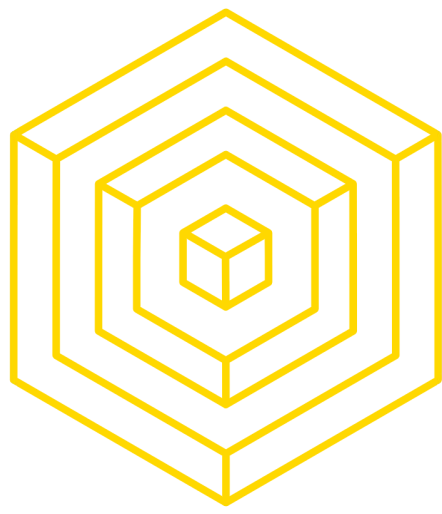
- Transaction: 0
- Listen on all transactions: ☐
- Filter with transaction hash or address:
- Transaction status: ☒ Success
- Transaction details: [block:9225020 txIndex:11] from: 0xda8...b9d70 to: Attendance.addStudent(string) 0x376...c2f0e value: 0 wei data: 0x453...00000 logs: 0 hash: 0x342...11644
- Buttons: [view on Etherscan](#), [view on Blockscout](#), [Debug](#)

The bottom status bar includes a 'Scam Alert' icon, a link to 'Initialize as git repo', and a 'Did you know?' message: 'You can use 'Generate documentation' in the right-click menu to get AI-generated documentation.'

AUTHOR:OLIVIA LI



AUTHOR:OLIVIA LI



Look at deployment on Etherscan

The screenshot displays the REMIX IDE v0.71.0 interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' sidebar is active, showing the 'Injected Provider - MetaMask' environment on the 'Sepolia (11155111) network'. The account '0xDA8...b9d70' is selected with a balance of 0.093422... ETH. The gas limit is set to 'Estimated Gas' with a custom value of 3,000,000. The value is 0 Wei. The contract 'Attendance - contracts/attendance' is selected, and the 'Deploy' button is highlighted. Below the sidebar, it shows 'Transactions recorded 6' and 'Deployed Contracts 2'.

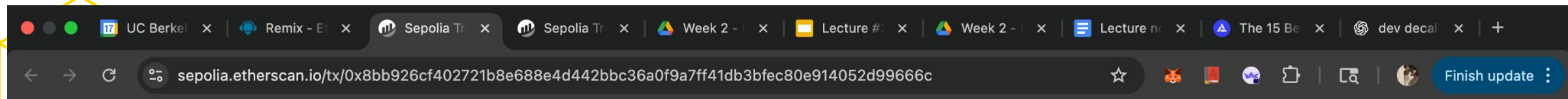
The main editor shows the Solidity code for the 'Attendance' contract. The code includes a dynamic array of student names, a function to get the number of students, and a function to get the full list of student names. A comment indicates that the count is updated automatically.

The bottom panel shows the transaction details for the deployment. A yellow box highlights the transaction information:

```
[block:9225020 txIndex:11] from: 0xda8...b9d70  
to: Attendance.addStudent(string) 0x376...c2f0e value: 0 wei  
data: 0x453...00000 logs: 0 hash: 0x342...11644
```

Buttons for 'view on Etherscan' and 'view on Blockscout' are visible above the transaction details. A 'Debug' button is also present.

AUTHOR: OLIVIA LI



Sepolia Testnet

Search by Address / Txn Hash / Block / Token

/

Finish update

Etherscan

HomeBlockchainTokensNFTsMore

Transaction Details

<>

OverviewState

</> API

TRANSACTION ACTION

Call 0x60806040 Method by 0xDA826775...E879b9d70

[This is a Sepolia Testnet transaction only]

Transaction Hash:

0x8bb926cf402721b8e688e4d442bbc36a0f9a7ff41db3bfec80e914052d99666c

Status:

Success

Block:

92250023 Block Confirmations

Timestamp:

36 secs ago (Sep-17-2025 11:31:24 PM UTC)

From:

0xDA8267757D54A0cD47125677A8285B1E879b9d70

To:

[0x3761fd18b91b90fbaec26aa6a9dd4f91dbc2f0e Created]

Value:

0 ETH

Transaction Fee:

0.000716890797957612 ETH

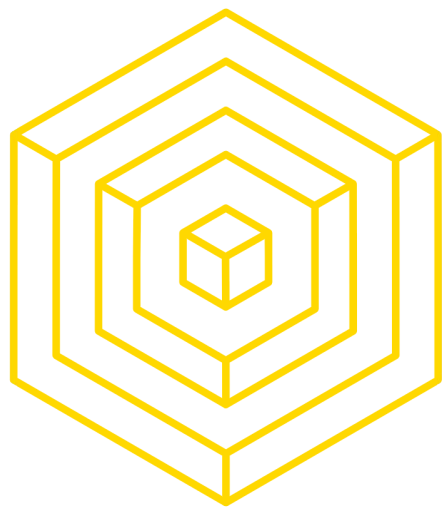
Gas Price:

1.500003762 Gwei (0.000000001500003762 ETH)

More Details:

Click to show more

AUTHOR: OLIVIA LI



Test Functions

34

REMIX v0.71.0

attendance

DEPLOY & RUN TRANSACTIONS

Attendance - contracts/attendance

evm version: prague

Deploy

Publish to IPFS

At Address Load contract from Address

Transactions recorded 5

Deployed Contracts 2

ATTENDANCETEST AT 0XCC

ATTENDANCE AT 0X376...C

Balance: 0 ETH

addStudent Olivia

getNumberOf...

getStudentNa...

Low level interactions

CALLDATA

Transact

Compiled

1 // SPDX-License-Identifier: GPL-3.0

2

3 pragma solidity >=0.7.0 <0.9.0;

4 import "remix_tests.sol"; // this import is automatically injected by Remix.

5 import "hardhat/console.sol";

6 import "../contracts/attendance.sol";

7

8

9 contract AttendanceTest {

10 Attendance attendance;

11

12 /// Called before each test

13 function beforeEach() public { infinite gas

14 attendance = new Attendance();

15 }

16

17 /// Test: contract starts with zero students

18 function testInitialStudentCountIsZero() public { infinite gas

19 Assert.equal(attendance.getNumberOfStudents(), uint(0), "Initial student count sh

20 }

21

22 /// Test: add one student

23 function testAddSingleStudent() public { infinite gas

24 attendance.addStudent("Alice");

25 }

Explain contract

AI copilot

0 Listen on all transactions

Filter with transaction hash or ad...

transact to Attendance.addStudent pending ...

transact to Attendance.addStudent errored: Error occurred: [object Object].

[object Object]

If the transaction failed for not having enough gas, try increasing the gas limit gently.

Scam Alert

Initialize as git repo

Did you know? You can use 'Generate documentation' in the right-click menu to get AI-generated documentation.

AUTHOR:OLIVIA LI

MetaMask

To exit full screen, press and hold esc



Account 1
Sepolia



Transaction request

Estimated changes ?

No
changes

Request from ?

remix.ethereum.org

Interacting with Alert >

0x3761f...C2F0e

Network fee ?

0.0001 SepoliaETH

Speed

Market ~12 sec

Cancel

Confirm

AUTHOR:OLIVIA LI



Look at transaction on Etherscan

The screenshot displays the Remix IDE interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' sidebar shows the 'Attendance' contract at address 0x376...c2f0e. The main editor shows the Solidity code for the 'AttendanceTest' contract, which includes a 'beforeEach' function and two test functions: 'testInitialStudentCountIsZero' and 'testAddSingleStudent'. The bottom panel shows the transaction details for the 'addStudent' function, with a yellow box highlighting the transaction hash and details: '[block:9225020 txIndex:11] from: 0xda8...b9d70 to: Attendance.addStudent(string) 0x376...c2f0e value: 0 wei'. The transaction status is 'Success'.

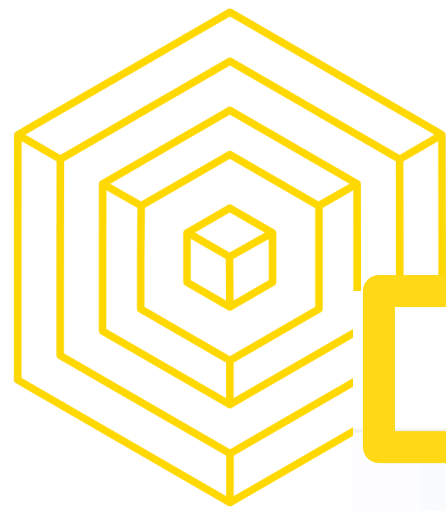
```
1 // SPDX-License-Identifier: GPL-3.0
2
3 pragma solidity >=0.7.0 <0.9.0;
4 import "remix_tests.sol"; // this import is automatically injected by Remix.
5 import "hardhat/console.sol";
6 import "../contracts/attendance.sol";
7
8 contract AttendanceTest {
9     Attendance attendance;
10
11     /// Called before each test
12     function beforeEach() public {
13         // infinite gas
14         attendance = new Attendance();
15     }
16
17     /// Test: contract starts with zero students
18     function testInitialStudentCountIsZero() public {
19         // infinite gas
20         Assert.equal(attendance.getNumberOfStudents(), uint(0), "Initial student count should be 0");
21     }
22
23     /// Test: add one student
24     function testAddSingleStudent() public {
25         // infinite gas
26         attendance.addStudent("Alice");
27     }
28 }
```

view on Etherscan view on Blockscout

[block:9225020 txIndex:11] from: 0xda8...b9d70
to: Attendance.addStudent(string) 0x376...c2f0e value: 0 wei

Debug

AUTHOR: OLIVIA LI



Sepolia
Testnet

Sepolia Testnet

Search by Address / Txn Hash / Block / Token

From: 0x0Da8267757D54A0cD47125677A8285B1E879b9d70

To: 0x3761fd18B91b90fbaec26Aa6a9dD4f91dBC2F0e

Value: 0 ETH

Transaction Fee: 0.00010107017323398 ETH

Gas Price: 1.500002571 Gwei (0.000000001500002571 ETH)

Gas Limit & Usage by Txn: 68,230 | 67,380 (98.75%)

Gas Fees: Base: 0.000002571 Gwei | Max: 1.500003667 Gwei | Max Priority: 1.5 Gwei

Burnt & Txn Savings Fees:
Burnt: 0.00000000017323398 ETH (\$0.00)
Txn Savings: 0.00000000007384848 ETH (\$0.00)

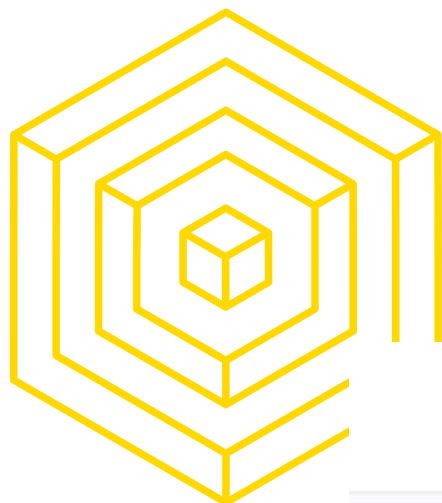
Other Attributes: Txn Type: 2 (EIP-1559) Nonce: 16 Position In Block: 11

Input Data:
Function: addStudent(string _id) ***
MethodID: 0x453e056a
[0]: 0020
[1]: 0006
[2]: 4f6c6976696100

View Input As Decode Input Data View In Decoder

More Details: [Click to show less](#)

A transaction is a cryptographically signed instruction that changes the blockchain state. Block explorers track the details of all transactions in the network. Learn more about transactions in our [Knowledge Base](#).
<https://sepolia.etherscan.io/inputdatadecoder?tx=0xb8342bb1f0208907f0ea759...>



Look at Functino called and input

Sepolia Testnet

Search by Address / Txn Hash / Block / Token

CSV Export

Account Balance Checker

Token Supply Checker

Token Standard Checker

Input Data Decoder

Similar Contract Search

Vyper Online Compiler

Bytecode to Opcode

Broadcast Transaction

Unit Converter

Base64 Converter

Block & Date Converter

UTF-8 Converter

Method ID Converter

Input Data Decoder Beta

Interpret and analyze data sent to Ethereum smart contracts.

Choose option:

☒ from Transaction ☐ from Address ☐ from ABI ☐ without ABI

Txn Hash

0xb8342bb1f0208907f0ea75934052d93ca4bed84eb8f8dc1bcdd2f019b70ea245



Decode

Reset

Decoded Results: 1 result found

③ Displaying functions that fit the entered input data, results below may not necessarily match the actual executed contract interaction.

Function addStudent

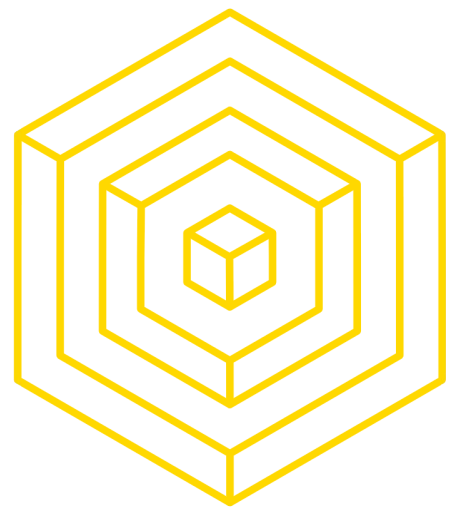
string _id

Olivia



Copy All Parameters





DEMO TIME!





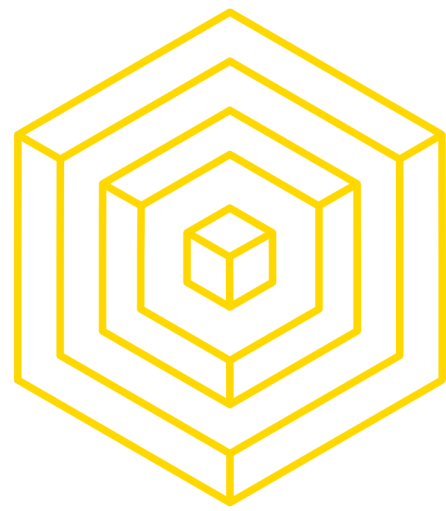
3 TESTNETS & RPC



TESTNETS

ALTERNATE REALITIES

- What problems might we run up against when trying to test our contract on Ethereum?



TESTNETS

ALTERNATE REALITIES

- What problems might we run up against when trying to test our contract on Ethereum?
 - **Immutability:** Bugs and mistakes cannot be changed once we deploy the contract.
 - **Gas Fees:** Transactions cost real-world money, which would make it expensive to test our contracts



TESTNETS

ALTERNATE REALITIES

The solution to these problems are **testnets**.

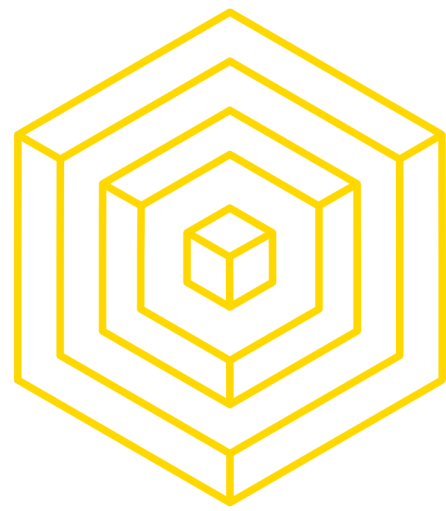
- A testnet is an alternative instance of a blockchain that is solely used for development.
 - Ethereum testnets run the EVM, smart contracts work exactly the same
 - The tokens (including the native asset such as ETH) have no value
 - “Faucets” give out ETH and other tokens for free



TESTNETS

ALTERNATE REALITIES

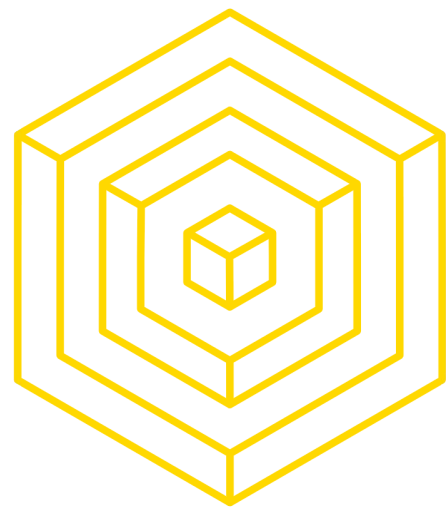
- Another option is to **test locally** by using a **local chain**
- Tools such as Foundry and HardHat let you spin up a local blockchain, providing you with public/private keypairs loaded with ETH for you to use.
- Local chains can even be forks of live chains such as Ethereum so that you can access existing contracts



REMOTE PROCEDURAL CALLS

TELLING BLOCKCHAINS TO DO STUFF

- **RPC** stands for:
 - **Remote**
 - **Procedural**
 - **Call**
- RPC calls are used to tell another computer to do something
- To deploy a smart contract, you must submit a transaction to a node, which will then submit it to the mempool. This node can either be:
 - Your own node running on your computer
 - A node running on another computer



RPC PROVIDERS

TELLING BLOCKCHAINS TO DO STUFF

- Companies that provide access to their node for you to interact with the blockchain are called **RPC Providers**
- Examples of RPC Providers for Ethereum:
 - Infura
 - Alchemy
 - Etherscan
- RPC Providers generally give you an API key with which you can send requests to their server.



4

SOLIDITY OVERVIEW





HISTORY OF SOLIDITY

- When Ethereum was proposed in 2013, the vision was a “world computer” where anyone could deploy decentralized programs (smart contracts). Known as the EVM!
- The EVM was designed as the runtime environment for these programs. But the EVM only understands bytecode — raw instructions.
- To make programming easier, developers designed Solidity (around 2014, led by Gavin Wood, Christian Reitwiessner, and others) as a high-level, human-readable language that compiles down to EVM bytecode.



4.1 DATA TYPES



CATEGORIES OF TYPES

DATA TYPES

VALUE TYPES

Passed by value

REFERENCE TYPES

Passed by reference

Arrays

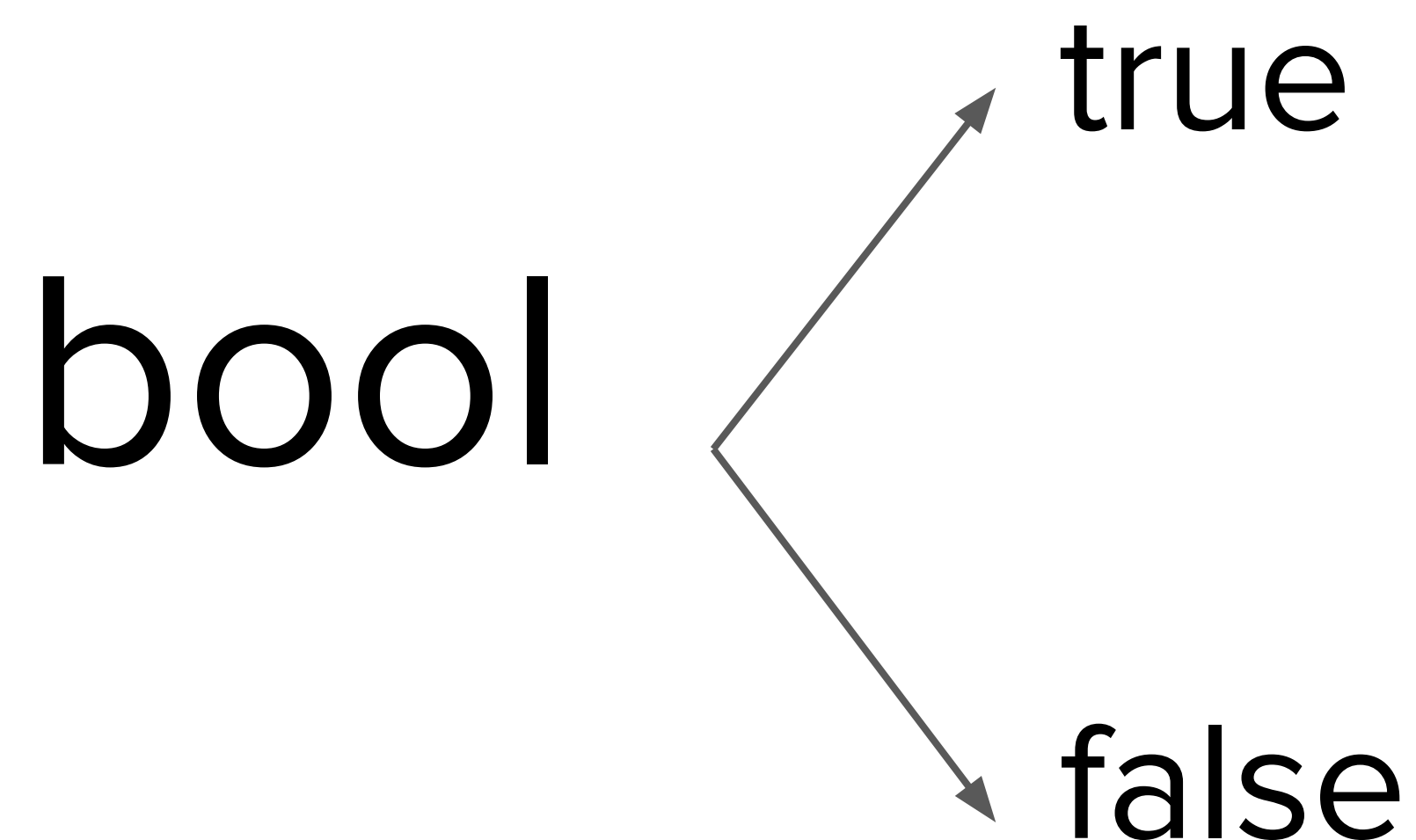
Structs

Data
Locations



BOOLEANS

VALUE TYPES



Operators:

- **!** (logical negation)
- **&&** (logical conjunction, “and”)
- **||** (logical disjunction, “or”)
- **==** (equality)
- **!=** (inequality)

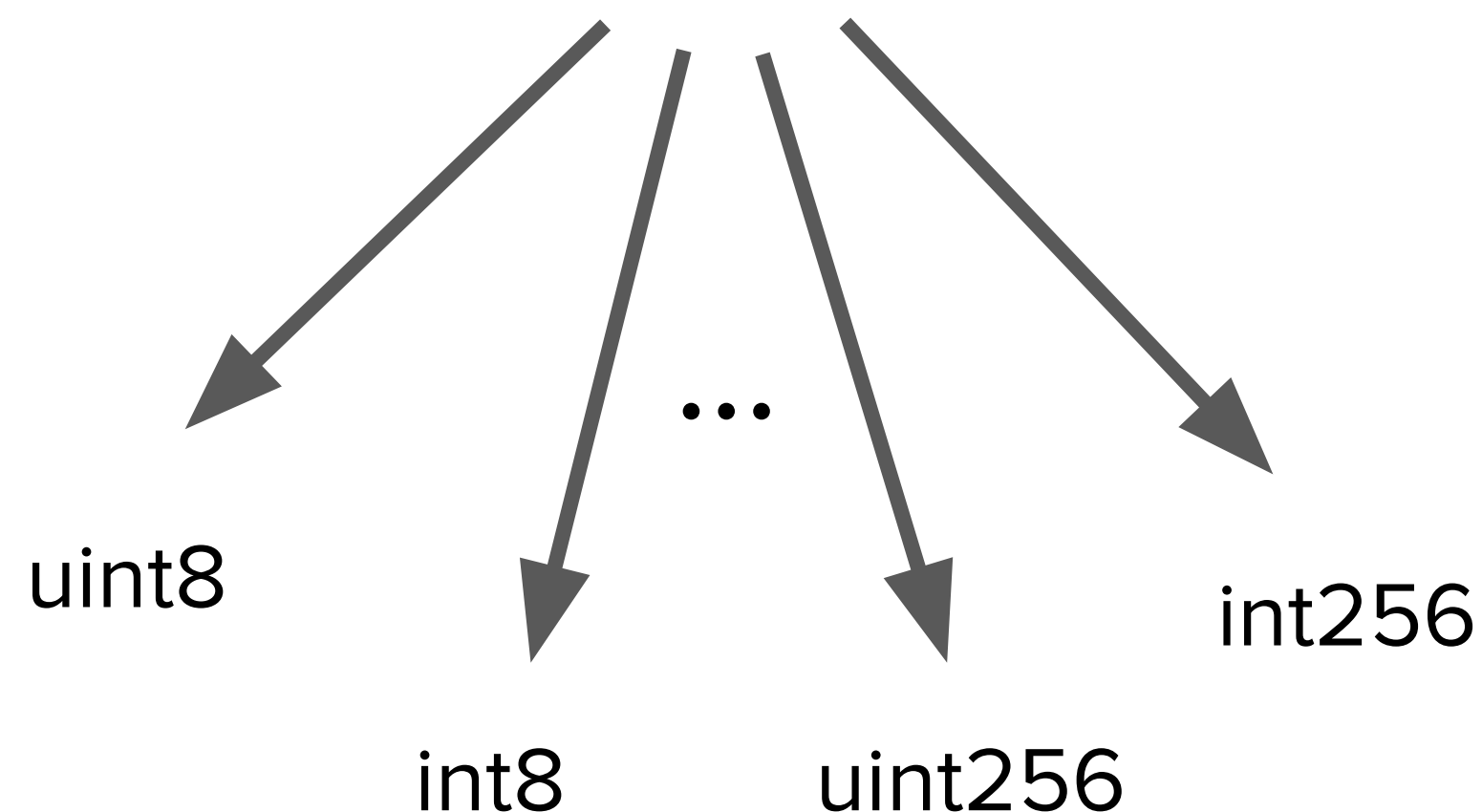


INTEGERS

VALUE TYPES

52

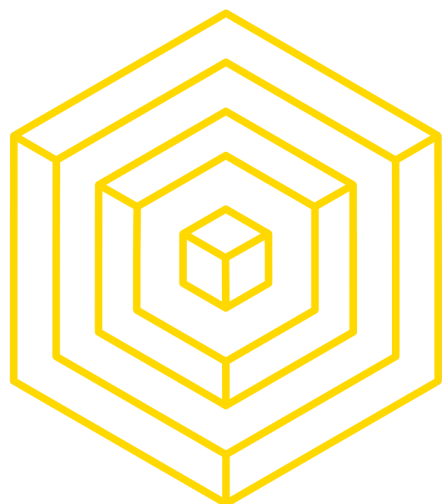
Integers



- int/uint ranges from int8/uint8 to int256/uint256.(In steps of 8)
- Int/uint are alias of int256/uint256

Operators:

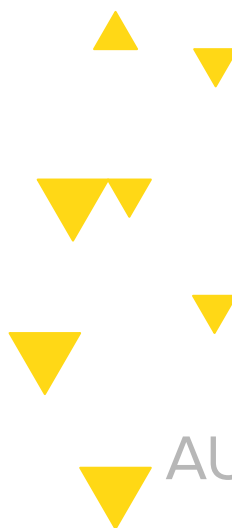
- Comparisons: `<=`, `<`, `==`, `!=`, `>=`, `>`
(evaluate to `bool`)
- Bit operators: `&`, `|`, `^` (bitwise exclusive or), `~` (bitwise negation)
- Shift operators: `<<` (left shift), `>>` (right shift)
- Arithmetic operators: `+`, `-`, unary `-`, `*`, `/`, `%` (modulo), `**` (exponentiation)



INTEGERS

VALUE TYPES

Unit	Wei Value	Wei
wei	1 wei	1
Kwei (babbage)	1e3 wei	1,000
Mwei (lovelace)	1e6 wei	1,000,000
Gwei (shannon)	1e9 wei	1,000,000,000
microether (szabo)	1e12 wei	1,000,000,000,000
milliether (finney)	1e15 wei	1,000,000,000,000,000
ether	1e18 wei	1,000,000,000,000,000,000





INTEGERS

VALUE TYPE

54

Bits Operation

(Performed on Two's Complement)

$\sim \text{int256}(0) == \text{int256}(-1)$

Shifts

$x \ll y == x * 2^{**}y$

$x \gg y == x / 2^{**}y$



INTEGERS

VALUE TYPE

Addition / Subtraction

(Performed on usual semantics)

$$\text{uint256}(0) - \text{uint256}(1) == 2^{**}256 - 1$$

Multiplication / Division

(Performed on usual semantics)

$$\text{int256}(-5) / \text{int256}(2) == \text{int256}(-2)$$



ADDRESS VALUE TYPE

Address & Address Payable

- Address: Holds a 20 byte value (size of an Ethereum address).
- Address Payable: Same as **address**, but with the additional members **transfer** and **send**.

Operators:

```
pragma solidity ^0.5.0;

contract WithdrawalContract {
    address public richest;
    uint public mostSent;

    mapping (address => uint) pendingWithdrawals;

    constructor() public payable {
        richest = msg.sender;
        mostSent = msg.value;
    }

    function becomeRichest() public payable returns (bool) {
        if (msg.value > mostSent) {
            pendingWithdrawals[richest] += msg.value;
            richest = msg.sender;
            mostSent = msg.value;
            return true;
        } else {
            return false;
        }
    }

    function withdraw() public {
        uint amount = pendingWithdrawals[msg.sender];
        // Remember to zero the pending refund before
        // sending to prevent re-entrancy attacks
        pendingWithdrawals[msg.sender] = 0;
        msg.sender.transfer(amount);
    }
}
```




ADDRESS PAYABLE

VALUE TYPE

Transfer & Send

```
address payable x = address(0x123);  
address myAddress = address(this);  
if (x.balance < 10 &&  
    myAddress.balance >= 10)  
    x.transfer(10);
```

The **transfer** function fails if the balance of the current contract is not large enough or if the Ether transfer is rejected by the receiving account. The **transfer** function reverts on failure.

Send is the low-level counterpart of **transfer**. If the execution fails, the current contract will not stop with an exception, but **send** will return **false**.



CONTRACT TYPES

VALUE TYPE

```
pragma solidity ^0.5.0;

contract D {
    uint public x;
    constructor(uint a) public payable {
        x = a;
    }
}

contract C {
    D d = new D(4); // will be executed as part of C's
    constructor

    function created(uint arg) public {
        D newD = new D(arg);
        newD.x();
    }

    function createAndEndowD(uint arg, uint amount)
    public payable {
        // Send ether along with the creation
        D newD = (new D).value(amount)(arg);
        newD.x();
    }
}
```

- Contracts do not support any operators.
- The members of contract types are the external functions of the contract including public state variables.
- For a contract **C** you can use **type(C)** to access type information about the contract.



STRING LITERALS

VALUE TYPES

```
bytes32 a = "stringliteral"
```

```
string b = "stringliteral"
```

```
a == b
```

```
Hex "DEADBEEF"
```

```
"\n\"'\\"abc\
```

```
def"
```

- `\<newline>` (escapes an actual newline)
- `\\` (backslash)
- `'` (single quote)
- `"` (double quote)
- `\b` (backspace)
- `\f` (form feed)
- `\n` (newline)
- `\r` (carriage return)
- `\t` (tab)
- `\v` (vertical tab)
- `\xNN` (hex escape))
- `\uNNNN` (unicode escape))



ENUM VALUE TYPES

```
pragma solidity >=0.4.16 <0.6.0;
```

```
contract test {  
    enum ActionChoices { GoLeft, GoRight,  
GoStraight, SitStill }  
    ActionChoices choice;  
    ActionChoices constant defaultChoice =  
ActionChoices.GoStraight;
```

```
    function setGoStraight() public {  
        choice = ActionChoices.GoStraight;  
    }
```

*// Since enum types are not part of the ABI, the signature of
"getChoice"*

// will automatically be changed to "getChoice() returns (uint8)"

// for all matters external to Solidity. The integer type used is

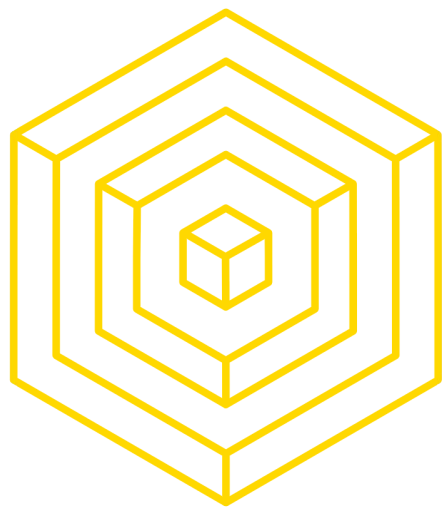
just

*// large enough to hold all enum values, i.e. if you have more
than 256 values,*

// `uint16` will be used and so on.

```
function getChoice() public view returns (ActionChoices) {  
    return choice;  
}
```

```
function getDefaultChoice() public pure returns (uint) {  
    return uint(defaultChoice);  
}  
}
```



PRECEDENCE

VALUE TYPES

Precedence	Description	Operator			
1	Postfix increment and decrement	<code>++</code> , <code>--</code>	5	Addition and subtraction	<code>+</code> , <code>-</code>
	New expression	<code>new <typename></code>	6	Bitwise shift operators	<code><<</code> , <code>>></code>
	Array subscripting	<code><array>[<index>]</code>	7	Bitwise AND	<code>&</code>
	Member access	<code><object>.<member></code>	8	Bitwise XOR	<code>^</code>
	Function-like call	<code><func>(<args...>)</code>	9	Bitwise OR	<code> </code>
	Parentheses	<code>(<statement>)</code>	10	Inequality operators	<code><</code> , <code>></code> , <code><=</code> , <code>>=</code>
2	Prefix increment and decrement	<code>++</code> , <code>--</code>	11	Equality operators	<code>==</code> , <code>!=</code>
	Unary plus and minus	<code>+</code> , <code>-</code>	12	Logical AND	<code>&&</code>
	Unary operations	<code>delete</code>	13	Logical OR	<code> </code>
	Logical NOT	<code>!</code>	14	Ternary operator	<code><conditional> ? <if-true> : <if-false></code>
	Bitwise NOT	<code>~</code>	15	Assignment operators	<code>=</code> , <code> =</code> , <code>^=</code> , <code>&=</code> , <code><<=</code> , <code>>>=</code> , <code>+=</code> , <code>-=</code> , <code>*=</code> , <code>/=</code>
3	Exponentiation	<code>**</code>	16	Comma operator	<code>,</code>
4	Multiplication, division and modulo	<code>*</code> , <code>/</code> , <code>%</code>			



4.2 DATA STRUCTURES

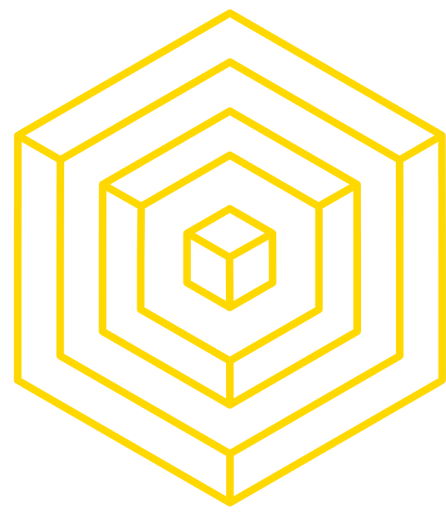


DATA STRUCTURES

ARRAYS

```
// Arrays
bytes32[5] nicknames; // static array
bytes32[] names; // dynamic array
uint newLength = names.push("John"); // adding returns new length of the array
// Length
names.length; // get length
names.length = 1; // lengths can be set (for dynamic arrays in storage only)

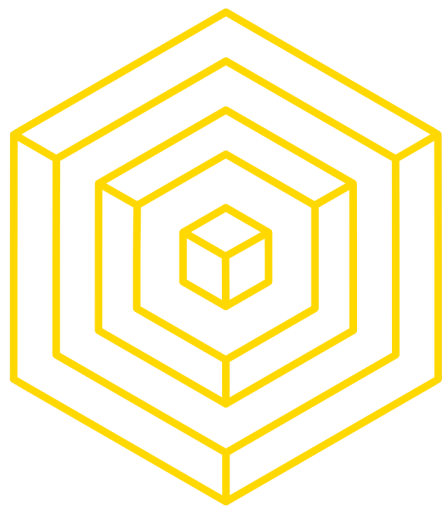
// multidimensional array
uint x[][5]; // arr with 5 dynamic array elements (opp order of most languages)
```



DATA STRUCTURES

STRUCTS

```
struct Bank {  
    address owner;  
    uint balance;  
}  
Bank b = Bank({  
    owner: msg.sender,  
    balance: 5  
});  
// or  
Bank c = Bank(msg.sender, 5);  
  
c.balance = 5; // set to new value  
delete b;  
// sets to initial value, set all variables in struct to 0, except mappings
```

DATA STRUCTURES

MAPPINGS

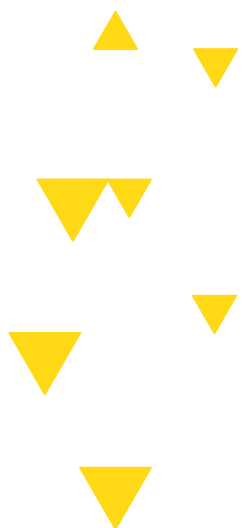
```
// Dictionaries (any type to any other type)
mapping (string => uint) public balances;
balances["charles"] = 1;
// balances["ada"] result is 0, all non-set key values return zeroes
// 'public' allows following from another contract
contractName.balances("charles"); // returns 1
// 'public' created a getter (but not setter) like the following:
function balances(string _account) returns (uint balance) {
    return balances[_account];
}
```

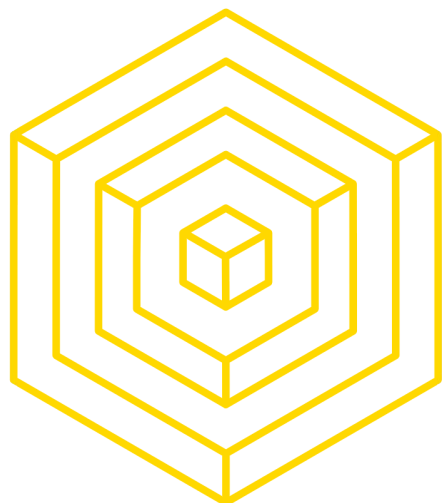



DATA STRUCTURES

MAPPINGS

```
// Nested mappings  
mapping (address => mapping (address => uint)) public custodians;  
  
// To delete  
delete balances["John"];  
delete balances; // sets all elements to 0
```





4.3 FUNCTIONS



Function basics

name **argType1 arg1, ...** **access classifier** **returnType (optional return var name)**

```
function withdraw(uint withdrawAmount) public returns (uint remainingBal) {  
    require(withdrawAmount <= balances[msg.sender]);  
  
    // Note the way we deduct the balance right away, before sending  
    // Every .transfer/.send from this contract can call an external function  
    // This may allow the caller to request an amount greater  
    // than their balance using a recursive call  
    // Aim to commit state before calling external functions, including .transfer/.send  
    balances[msg.sender] -= withdrawAmount;  
  
    // this automatically throws on a failure, which means the updated balance is reverted  
    msg.sender.transfer(withdrawAmount);  
  
    return balances[msg.sender];  
}
```

- public - all can access
- external - Cannot be accessed internally, only externally
- internal - only this contract and contracts deriving from it can access
- private - can be accessed only from this contract

IMPORTANT:

Access classifiers determine who can USE your functions, but everyone can see them, since they will be deployed on a public blockchain.



Payable functions

Only functions marked as payable can receive Ether

Non-payable functions with ether values will be routed to default function

```
function deposit() public payable returns (uint) {  
    // Use 'require' to test user inputs, 'assert' for internal invariants  
    // Here we are making sure that there isn't an overflow issue  
    require((balances[msg.sender] + msg.value) >= balances[msg.sender]);  
  
    balances[msg.sender] += msg.value;  
    // no "this." or "self." required with state variable  
    // all values set to data type's initial value by default  
  
    LogDepositMade(msg.sender, msg.value); // fire event  
  
    return balances[msg.sender];  
}
```



Fallback function

- Invoked when a function is called which does not match any other contract functions
- Only one per contract
- No arguments, returns nothing
- Typically payable - enables contract to receive ether sent directly to it

Can be thought of as default behavior when the contract does not recognize the command

```
function() external payable {  
    require(msg.value >= prize || msg.sender == owner);  
    king.transfer(msg.value);  
    king = msg.sender;  
    prize = msg.value;  
}
```




Constructors

- Create an instance of the contract with the given arguments
- Only one allowed, cannot be overloaded
- 2 implementations:
 - function [contractName] (arg1, arg2 ...)
 - constructor (arg1, arg2 ...)

```
contract SimpleBank { // CapWords
    // Declare state variables outside function, persist through life of contract

    // dictionary that maps addresses to balances
    // always be careful about overflow attacks with numbers
    mapping (address => uint) private balances;

    // "private" means that other contracts can't directly query balances
    // but data is still viewable to other parties on blockchain

    address public owner;
    // 'public' makes externally readable (not writeable) by users or contracts

    // Events - publicize actions to external listeners
    event LogDepositMade(address accountAddress, uint amount);

    // Constructor, can receive one or many variables here; only one allowed
    function SimpleBank() public {
        // msg provides details about the message that's sent to the contract
        // msg.sender is contract caller (address of contract creator)
        owner = msg.sender;
    }
}
```




6.4 WORKSHOP TIME



Attendance/Vitamin

